

CSE 4303

Data Structures

Topic: Hashing

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Hash table

- *Hash table* (or hash map) stores **key-value pairs** in an array-like format for fast data retrieval.
- Uses a *hash function* to compute an index used to store in the hash table or to find the value.
- Hash table provides average *$O(1)$ insertions and search.*

Hashing

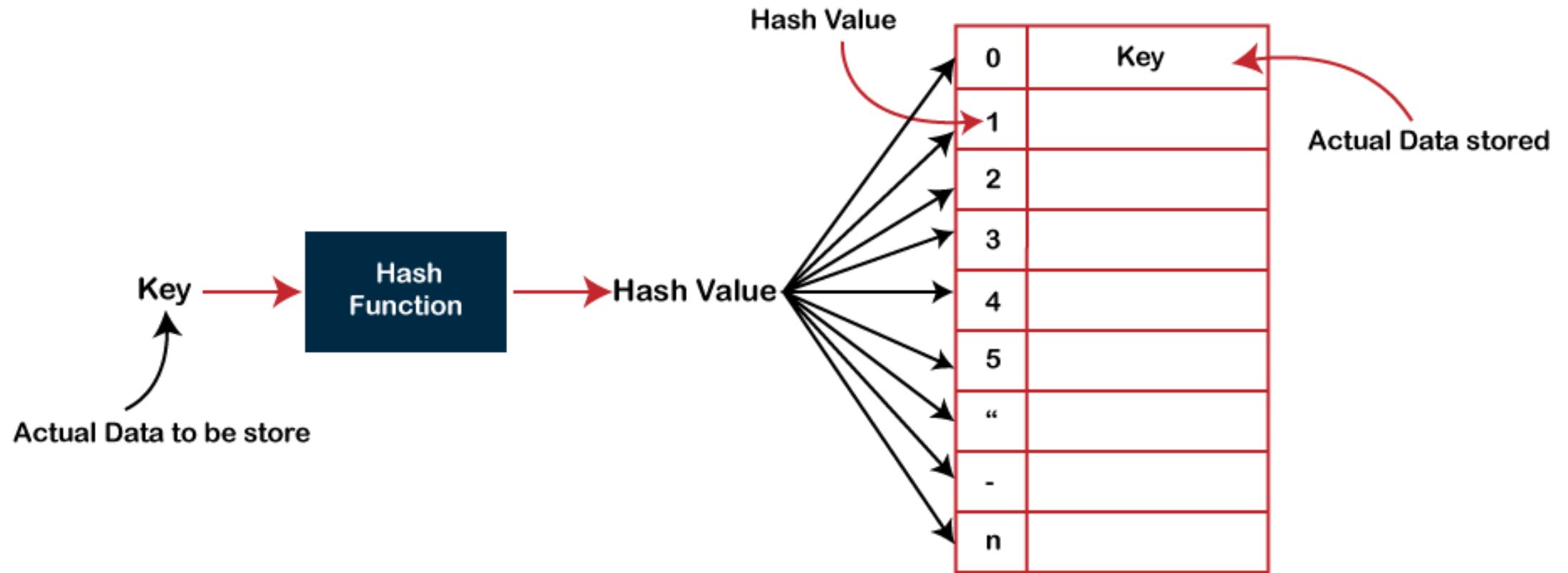
- *Hashing* is a technique that is used to uniquely identify a specific object from a group of similar objects.

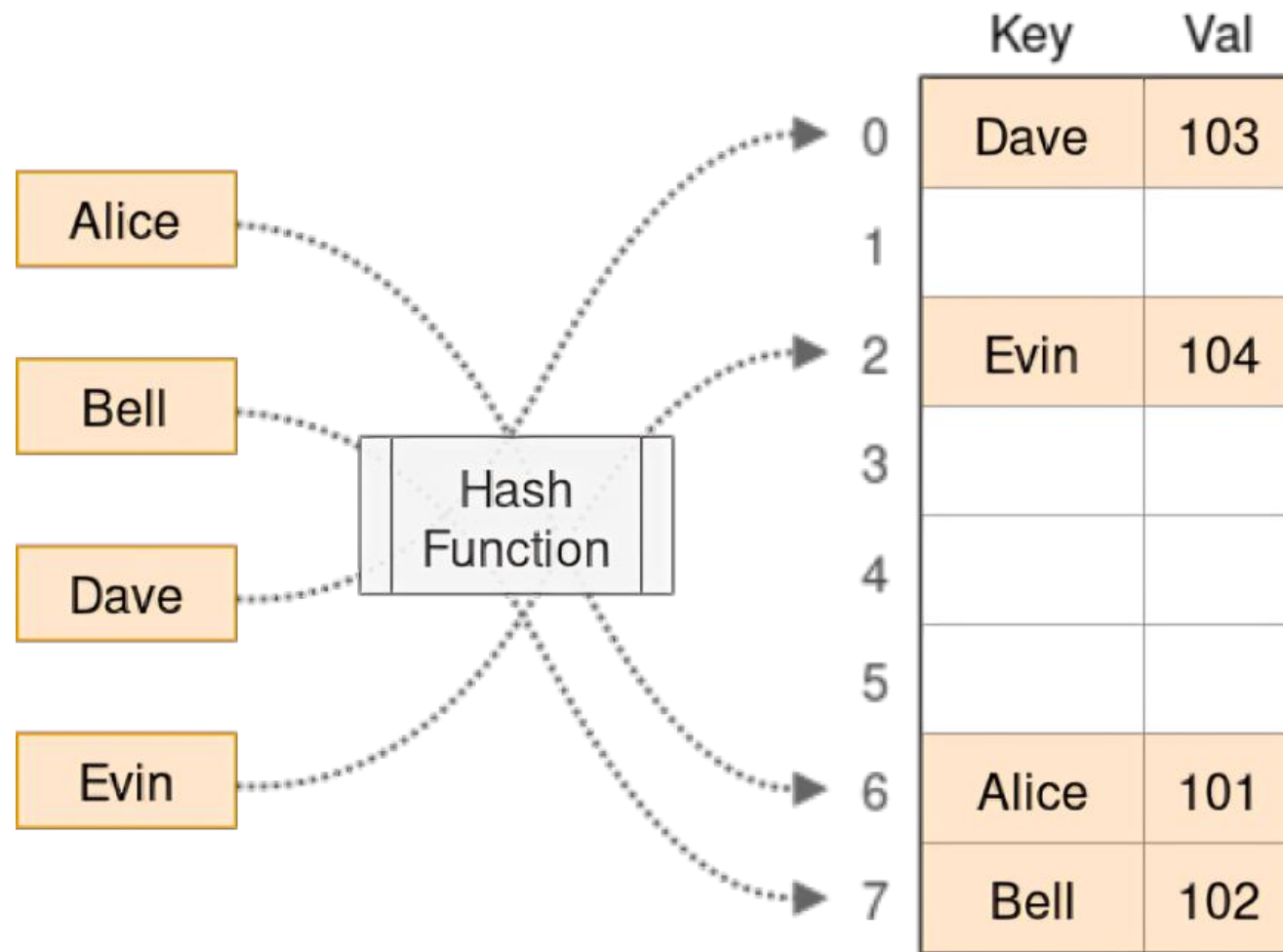
Hashing

- *Hashing* is a technique that is used to uniquely identify a specific object from a group of similar objects.
- In universities, each student is assigned a unique roll number that can be used to retrieve information about them.
- In libraries, each book is assigned a unique number that can be used to determine information about the book.

Hashing

- *Hashing* is a technique that is used to uniquely identify a specific object from a group of similar objects.
- In universities, each student is assigned a unique roll number that can be used to retrieve information about them.
- In libraries, each book is assigned a unique number that can be used to determine information about the book.
- To store the key/value pair, you can use a simple array like a data structure where keys can be used directly as an index to store values.
- *However, in cases where the keys are large and cannot be used directly as an index, you should use hashing.*





Hashing

- Hashing is a technique used to implement hash tables.

Insertions Deletions Searching	$O(1)$
findMax findMin Print data (the entire table) in sorted order	<i>Not great when ordering is required</i>

General Idea

- Hash Table is merely an array of fixed size containing the items.
 - size of the array/table as *TableSize*.
 - Items stored in the table are indexed by $0 \dots TableSize - 1$.

General Idea

- Hash Table is merely an array of fixed size containing the items.
 - size of the array/table as *TableSize*.
 - Items stored in the table are indexed by $0 \dots TableSize - 1$.
- Key is used to find the index of the associated item in the table.
 - Keys are mapped to numbers (i.e. index) ranging from $0 \dots TableSize - 1$.

General Idea

- Hash Table is merely an array of fixed size containing the items.
 - size of the array/table as *TableSize*.
 - Items stored in the table are indexed by $0 \dots TableSize - 1$.
- Key is used to find the index of the associated item in the table.
 - Keys are mapped to numbers (i.e. index) ranging from $0 \dots TableSize - 1$.
 - Key can be an integer/string/value of any type.
 - E.g. a Name/ID that is part of a large student database.

General Idea

- **Hash Table** is merely an array of fixed size containing the items.
 - size of the array/table as $TableSize$.
 - Items stored in the table are indexed by $0 \dots TableSize - 1$.
- Key is used to find the index of the associated item in the table.
 - Keys are mapped to numbers (i.e. index) ranging from $0 \dots TableSize - 1$.
 - Key can be an integer/string/value of any type.
 - E.g. a Name/ID that is part of a large student database.
- The mapping done using **Hash Function**.

Hash Functions

- Maps key to the Hash-table.
- Requirements:
 - Simple to compute (must satisfy $O(1)$!)

Hash Functions

- Maps key to the Hash-table.
- Requirements:
 - Simple to compute (must satisfy $O(1)$!)
 - Should minimize collisions (happens when multiple keys are mapped to the same index).

Hash Functions

- Maps key to the Hash-table.
- Requirements:
 - Simple to compute (must satisfy $O(1)$!)
 - Should minimize collisions (happens when multiple keys are mapped to the same index).
 - Distribute the keys evenly among the cells

Hash Functions

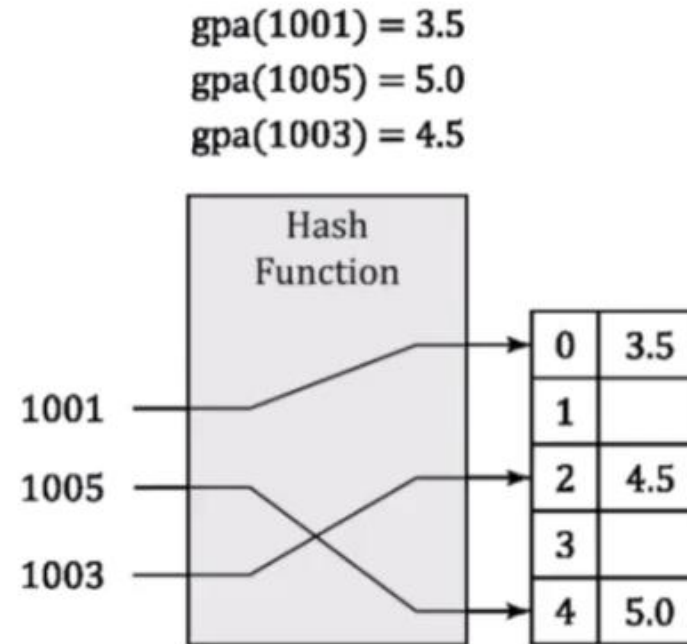
- Maps key to the Hash-table.
- **Requirements:**
 - Simple to compute (must satisfy $O(1)$!)
 - Should minimize collisions (happens when multiple keys are mapped to the same index).
 - Distribute the keys evenly among the cells
- If we know which keys will occur (in advance) we can design **perfect hash functions**, but we don't know!
- Need to *design hash functions to properly distribute* the items in the hash tables.

Designing Hash functions

- If keys are integers, $(key \% TableSize)$ is a general strategy.

```
index = hash_function(key)

Hash_function (key){
    index = key % TableSize
    return index -1;
}
```



- If the keys are strings, the Hash function needs more care.
 - First needed to be converted into a numeric value.

Hash Functions

- Challenges:
 - Keys may not be numeric.

Hash Functions

- Challenges:
 - Keys may not be numeric.
 - The number of possible keys are much larger than the space available in the table.

Hash Functions

- Challenges:
 - Keys may not be numeric.
 - The number of possible keys are much larger than the space available in the table.
 - Different keys may map into the same location (collision).
 - If there are too many collisions, the performance of hash-table

More Ideas to design Hash functions

- Truncation:
 - Key = 123456789, map to a table of 1000 addresses by picking 3 digits of the key

More Ideas to design Hash functions

- Truncation:
 - Key = 123456789, map to a table of 1000 addresses by picking 3 digits of the key
- Folding:
 - 123|456|789 : add the folds and take the mod. = $1368 \% 1000 = 368$

More Ideas to design Hash functions

- Truncation:
 - Key = 123456789, map to a table of 1000 addresses by picking 3 digits of the key
- Folding:
 - 123|456|789 : add the folds and take the mod. = $1368 \% 1000 = 368$
- Key mod N:
 - N is the size of the table. Better if N is prime (helps in collision handling).

More Ideas to design Hash functions

- Truncation:
 - Key = 123456789, map to a table of 1000 addresses by picking 3 digits of the key
- Folding:
 - 123|456|789 : add the folds and take the mod. = $1368 \% 1000 = 368$
- Key mod N:
 - N is the size of the table. Better if N is prime (helps in collision handling).
- Squaring:
 - Square the key and then truncate.

More Ideas to design Hash functions

- Add up the ASCII values of all characters of the key.

```
int hash(const string &key, int tableSize)
{
    int hasVal = 0;

    for (int i = 0; i < key.length(); i++)
        hashVal += key[i];
    return hashVal % tableSize;
}
```

- Simple to implement and fast.

Any observations on this function?

Hash Table

Here all strings are sorted at same index

Index				
0				
1				
2	abcdef	bcdefa	cdefab	defabc
3				
4				
-				
-				
-				
-				

More Ideas to design Hash functions

- Add up the ASCII values of all characters of the key.

```
int hash(const string &key, int tableSize)
{
    int hasVal = 0;

    for (int i = 0; i < key.length(); i++)
        hashVal += key[i];
    return hashVal % tableSize;
}
```

- Simple to implement and fast.

*Any observations on this
function?*

*Permutations of the same
string result in the same hash
value!*

More Ideas to design Hash functions

- Add up the ASCII values of all characters of the key.

```
int hash(const string &key, int tableSize)
{
    int hasVal = 0;

    for (int i = 0; i < key.length(); i++)
        hashVal += key[i];
    return hashVal % tableSize;
}
```

- Simple to implement and fast.
- Besides, if the *tableSize* is large, the function does not distribute keys well.
- Let, *tableSize* = 10000, *key_length* ≤ 8, Hash function can assume values only between 0 and 1016.

More Ideas to design Hash functions

- Let's try a different hash function.
- Index for a specific string = *sum of ASCII values of characters multiplied by their respective order* in the string after which it is modulo with 2069 (prime number).

More Ideas to design Hash functions

- Let's try a different hash function.
- Index for a specific string = *sum of ASCII values of characters multiplied by their respective order* in the string after which it is modulo with 2069 (prime number).

String	Hash value	Index
abcdef	$(97 \times 1 + 98 \times 2 + 99 \times 3 + 100 \times 4 + 101 \times 5 + 102 \times 6) \% 2069$	38

More Ideas to design Hash functions

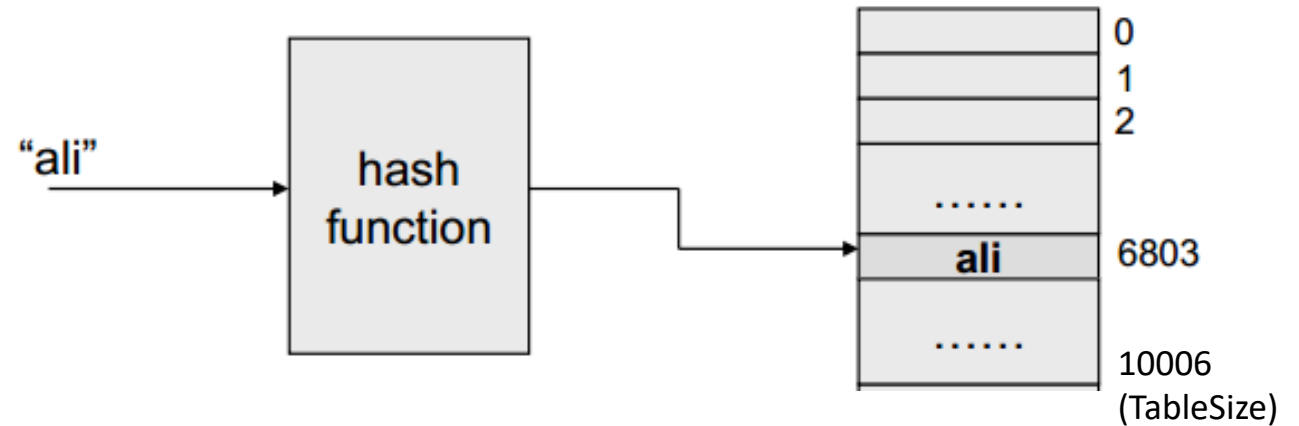
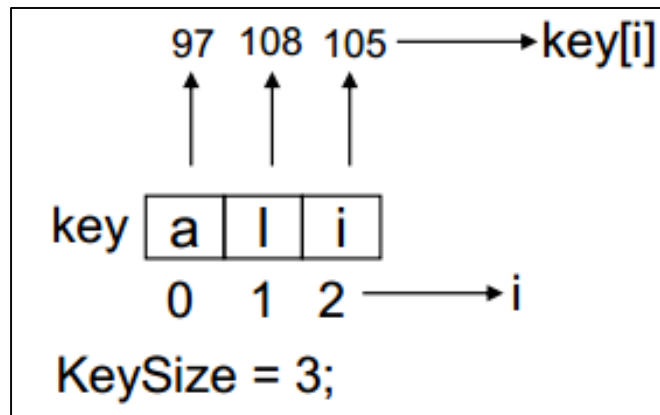
- Let's try a different hash function.
- Index for a specific string = *sum of ASCII values of characters multiplied by their respective order* in the string after which it is modulo with 2069 (prime number).

String	Hash value	Index
abcdef	$(97 \times 1 + 98 \times 2 + 99 \times 3 + 100 \times 4 + 101 \times 5 + 102 \times 6) \% 2069$	38
bcdefa	$(98 \times 1 + 98 \times 2 + 100 \times 3 + 101 \times 4 + 102 \times 5 + 97 \times 6) \% 2069$	23
cdefab	$(99 \times 1 + 100 \times 2 + 101 \times 3 + 102 \times 4 + 97 \times 5 + 98 \times 6) \% 2069$	14
defabc	$(100 \times 1 + 101 \times 2 + 102 \times 3 + 97 \times 4 + 98 \times 5 + 99 \times 6) \% 2069$	11

Here all strings are stored at different indices

Index	
0	
1	
-	
-	
-	
11	defabc
12	
13	
14	cdefab
-	
-	
-	
-	
23	bcdefa
-	
-	
-	
38	abcdef
-	
-	

$$\text{Hash}(\text{key}) = \sum_{i=0}^{\text{keySize}-1} \text{key}[\text{keySize} - i - 1] \times 37^i$$



$$\text{Hash}('ali') = 105 \times 37^0 + 108 \times 37^1 + 97 \times 37^2 = 6803$$

Collision resolution

- Collision: situations where two keys *hash to the same index*.
- Collision resolution techniques:
 1. Separate Chaining/ Open Hashing
 2. Open addressing/ Closed Hashing
 - i. Linear Probing
 - ii. Quadratic Probing
 - iii. Double Hashing

Separate chaining

- Idea: maintain individual lists for items that hash to same value.

Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

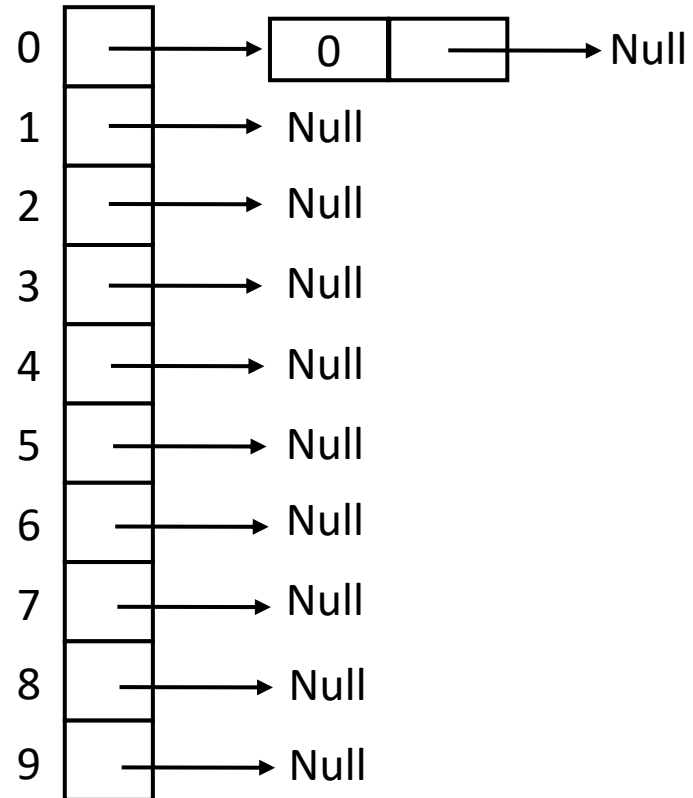
Keys: 0, 1, 4, 9, 16, 25,
36, 49, 64, 81

0		Null
1		Null
2		Null
3		Null
4		Null
5		Null
6		Null
7		Null
8		Null
9		Null

Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

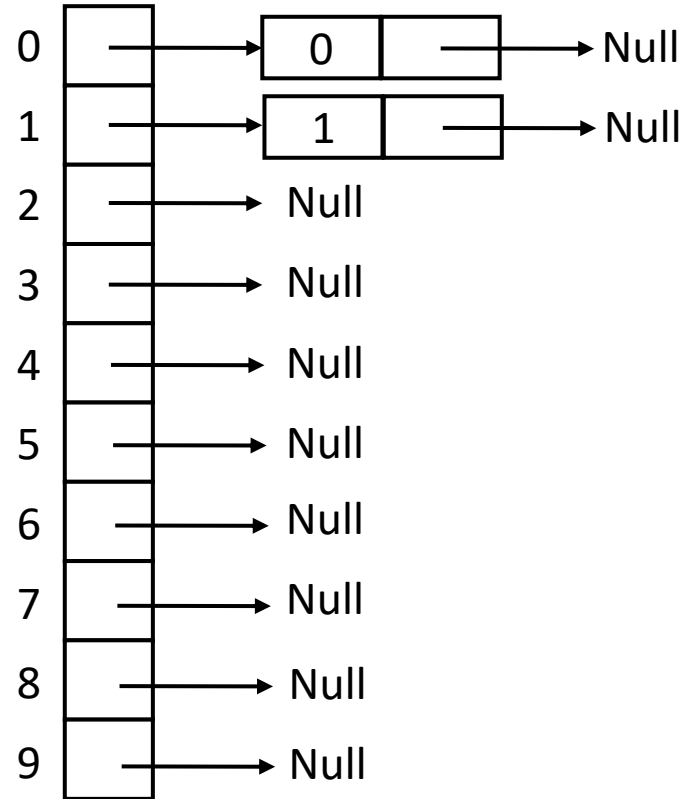
Keys: 0, 1, 4, 9, 16, 25,
36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

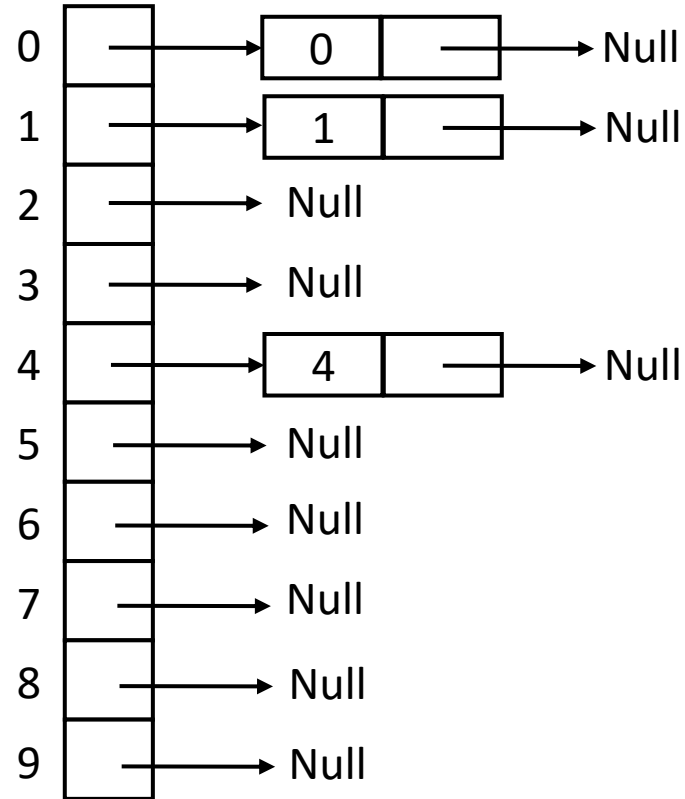
Keys: 0, 1, 4, 9, 16, 25,
36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

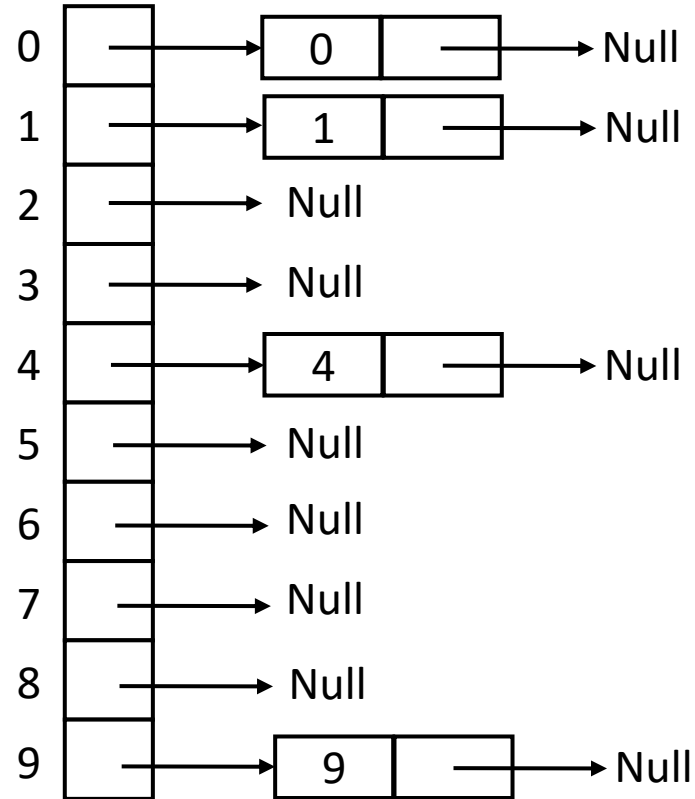
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

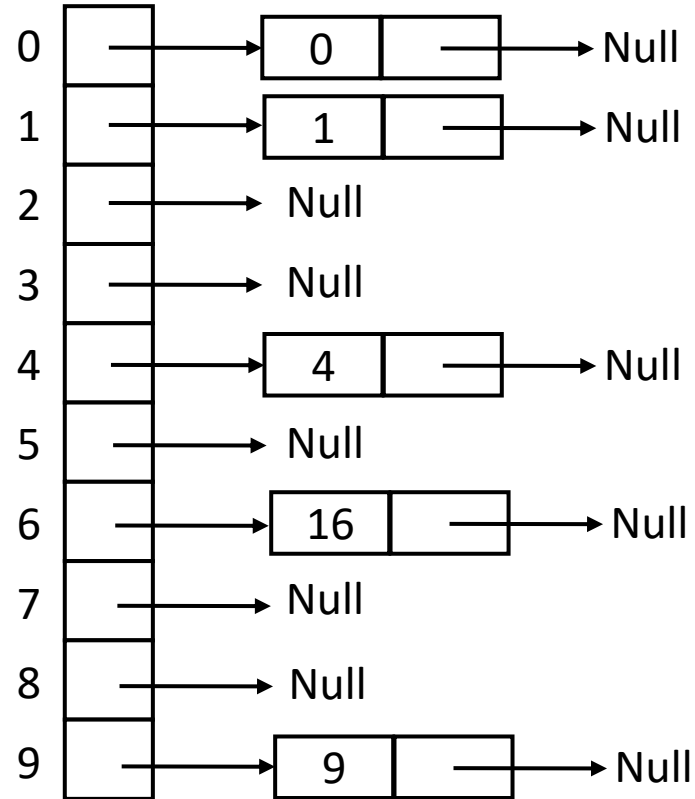
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

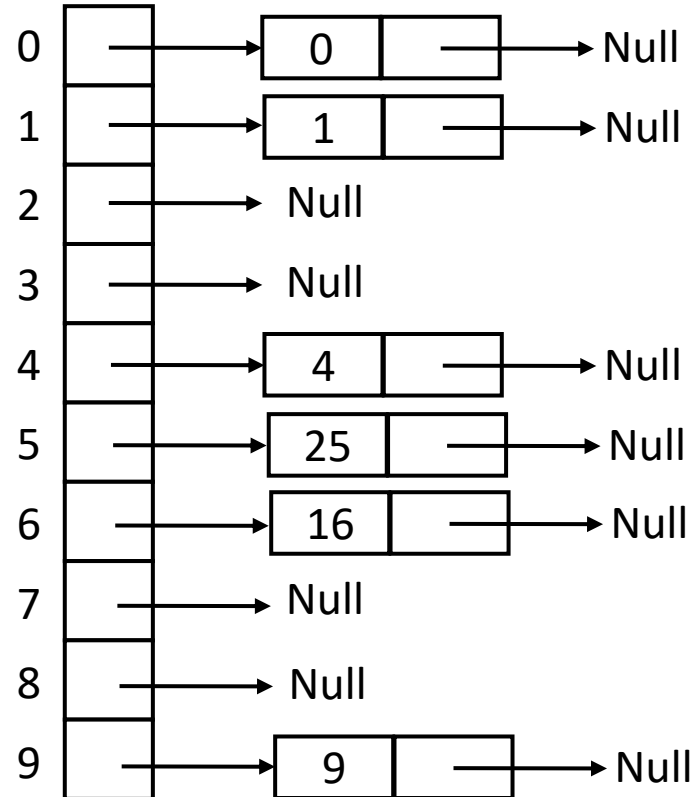
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

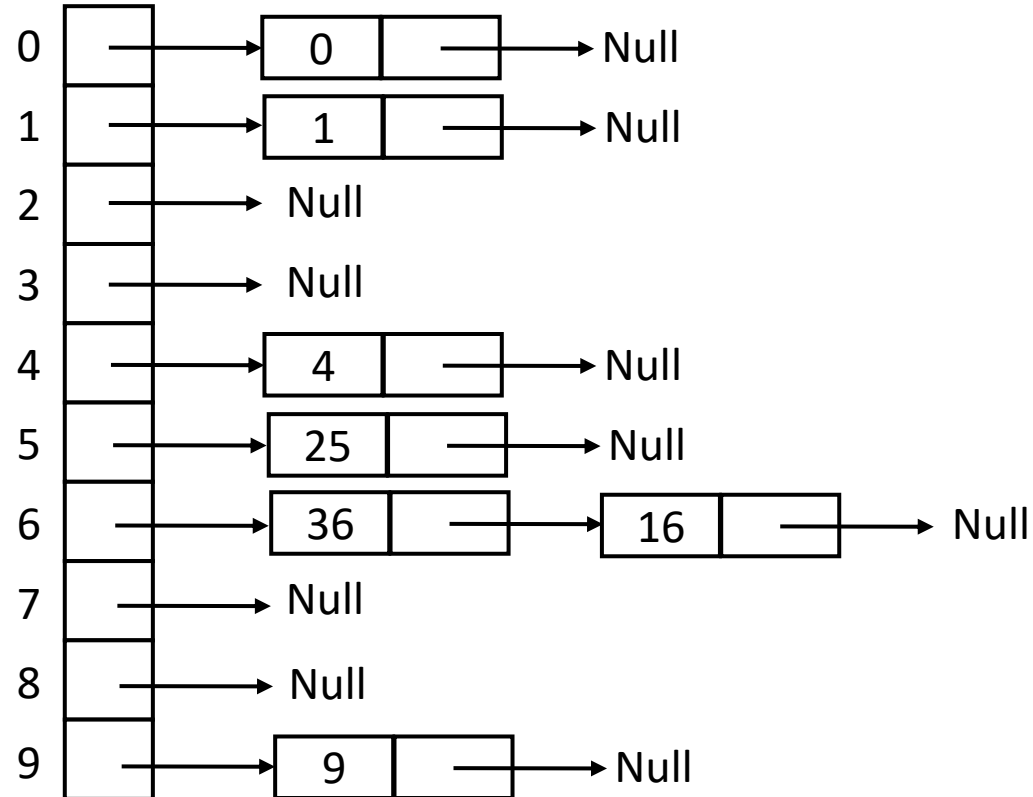
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

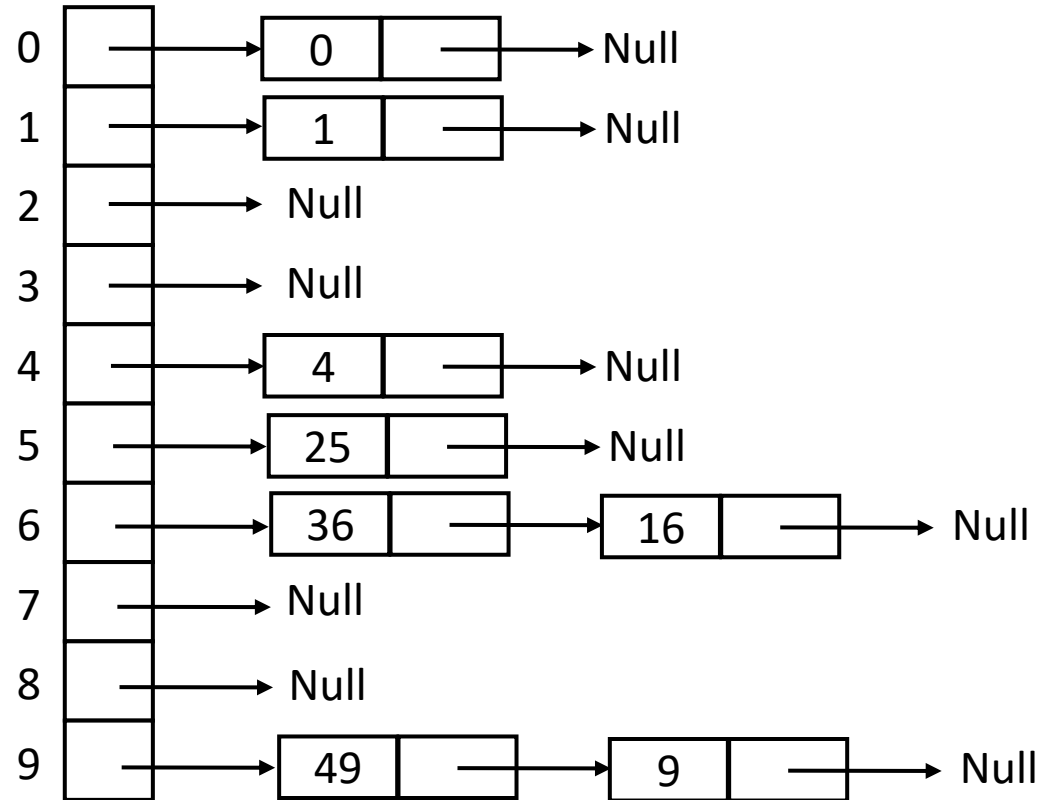
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

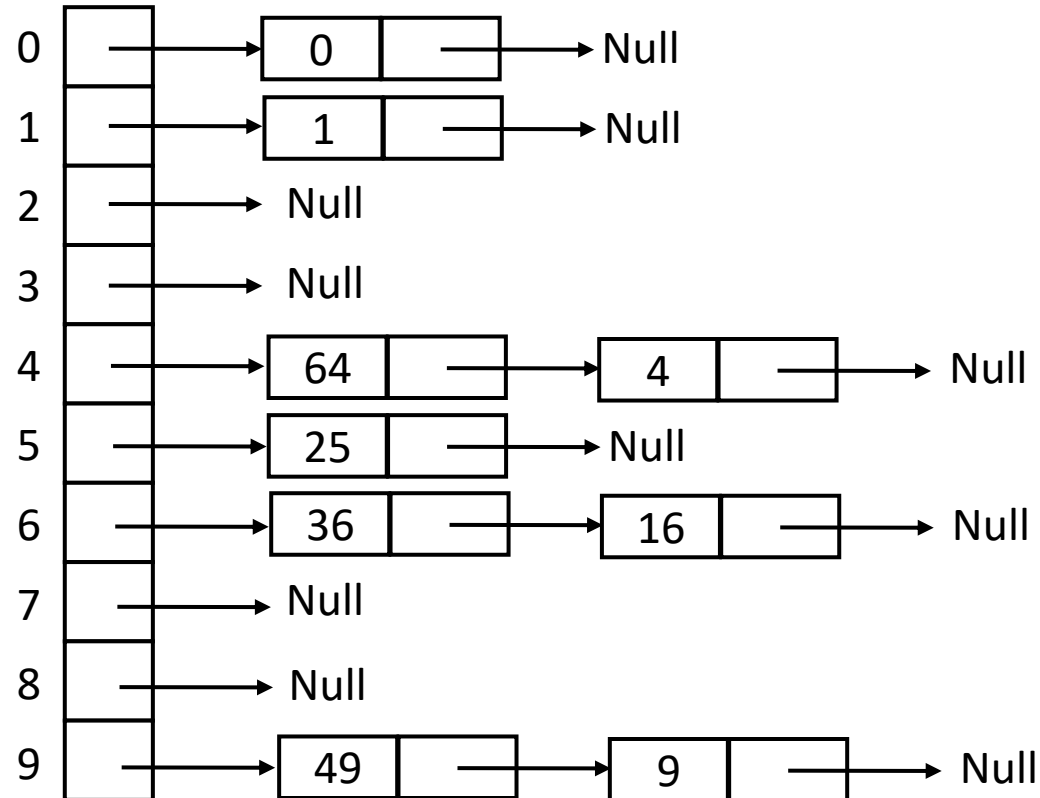
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

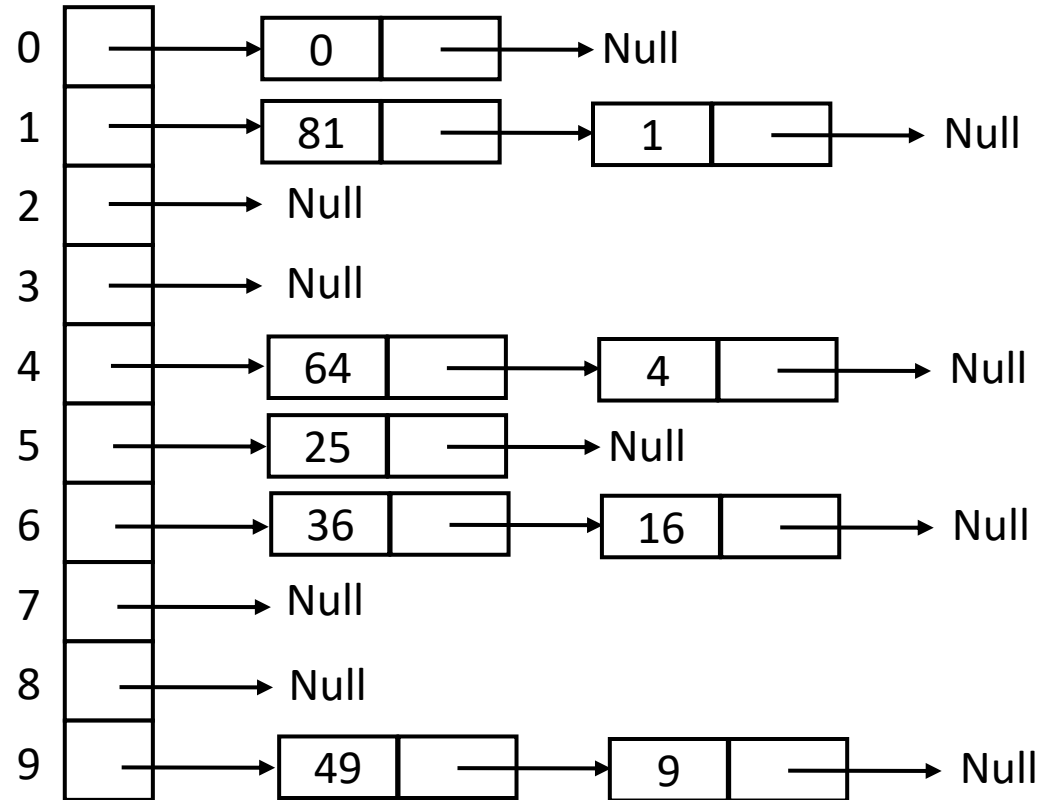
Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

$\text{hash}(\text{key}) = \text{key} \% 10$

Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81



Separate chaining

- Idea: maintain individual lists for items that hash to the same value.
 - What do the elements of the Hash table initially point to?

Separate chaining

- Idea: maintain individual lists for items that hash to the same value.
 - What do the elements of the Hash table initially point to?
 - To the first nodes (heads) of each list.
 - Shall we insert it in the front or back of the list?

Separate chaining

- Idea: maintain individual lists for items that hash to the same value.
 - What do the elements of the Hash table point to?
 - To the first nodes (heads) of each list.
 - Shall we insert it in the front or back of the list?
 - Inserting front is better: $O(1)$
 - Insert back is possible, but need to maintain a tail pointer for each list (costly).

Separate chaining

- Idea: maintain individual lists for items that hash to the same value.
 - What do the elements of the Hash table point to?
 - To the first nodes (heads) of each list.
 - Shall we insert it in the front or back of the list?
 - Inserting front is better: $O(1)$
 - Insert back is possible, but need to maintain a tail pointer for each list (costly).
- Advantages:
 - Simple to implement!
 - Can store more items than *TableSize*. (*Never runs out of space!*)

Separate Chaining (Operations)

- How to initialize the Hash Table?

Separate Chaining (Operations)

- How to initialize the Hash Table?
 - Set all entries to NULL.
- How to search:

Separate Chaining (Operations)

- How to initialize the Hash Table?
 - Set all entries to NULL.
- How to search:
 - Locate the cell using the Hash function.
 - Perform a sequential search on the Linked list in the cell.

Separate Chaining (Operations)

- **How to initialize the Hash Table?**
 - Set all entries to NULL.
- **How to search:**
 - Locate the cell using the Hash function.
 - Perform a sequential search on the Linked list in the cell.
- **Insertion:**
 - Locate the cell using the Hash function.
 - Insert it as the first item in the list.
- **Deletion:**
 - Locate the cell using the Hash function
 - Delete the item from the Linked list.

- Collisions are very likely.
- What is the average length of the lists?
 - Represented using Load factor (λ)

- Collisions are very likely.
- What is the average length of the lists?
 - Represented using Load factor (λ)
- Load factor (λ) = $\frac{\text{total number of items}}{\text{tableSize}}$
- What is the maximum load factor for separate chaining?

- Collisions are very likely.
- What is the average length of the lists?
 - Represented using Load factor (λ)
- Load factor (λ) =
$$\frac{\text{total number of items}}{\text{tableSize}}$$
- What is the maximum load factor for separate chaining?
 - For chaining, λ can be >1 .

Separate chaining(cost of searching)

- COST = $O(1)$ to evaluate the Hash function +
Time needed to traverse the List.

Separate chaining(cost of searching)

- COST = $O(1)$ to evaluate the Hash function +
Time needed to traverse the List.
- Unsuccessful search: $O(1 + \lambda)$
 - Need to traverse the entire list (compare λ nodes on average.)

Separate chaining(cost of searching)

- $\text{COST} = O(1)$ to evaluate the Hash function +
Time needed to traverse the List.
- Unsuccessful search: $O(1 + \lambda)$
 - Need to traverse the entire list (compare λ nodes on average.)
- Successful search:
 - On average, need to check half of the nodes while searching.
 - Average cost: $O(1 + \lambda/2)$

Chaining (Pros and cons)

- Pros:
 - Simple to implement
 - Hash-table never fills up. Can always add more items.
 - Less sensitive to the hash function and load factor.

Chaining (Pros and cons)

- Pros:
 - Simple to implement
 - Hash-table never fills up. Can always add more items.
 - Less sensitive to the hash function and load factor.
 - Widely used when it is unknown how many and how frequently keys may be inserted or deleted.

Chaining (Pros and cons)

- **Pros:**
 - Simple to implement
 - Hash-table never fills up. Can always add more items.
 - Less sensitive to the hash function and load factor.
 - Widely used when it is unknown how many and how frequently keys may be inserted or deleted.
- **Cons:**
 - Some parts of the hash table may never be used (bad distribution)

Chaining (Pros and cons)

- **Pros:**
 - Simple to implement
 - Hash-table never fills up. Can always add more items.
 - Less sensitive to the hash function and load factor.
 - Widely used when it is unknown how many and how frequently keys may be inserted or deleted.
- **Cons:**
 - Some parts of the hash table may never be used (bad distribution)
 - If the chain becomes long, search time becomes $O(N)$ in worst case.

Chaining (Pros and cons)

- **Pros:**
 - Simple to implement
 - Hash-table never fills up. Can always add more items.
 - Less sensitive to the hash function and load factor.
 - Widely used when it is unknown how many and how frequently keys may be inserted or deleted.
- **Cons:**
 - Some parts of the hash table may never be used (bad distribution)
 - If the chain becomes long, search time becomes $O(N)$ in worst case.
 - Use extra space for links.

Hashing: Open Addressing

Collision resolution with open addressing

- In Open addressing, all the items go inside the table.
 - No chain is maintained.

Collision resolution with open addressing

- In Open addressing, all the items go inside the table.
 - No chain is maintained.
- Bigger table may be needed.
 - Generally Load factor (λ) should be ≤ 0.5 .

Collision resolution with open addressing

- In Open addressing, all the items go inside the table.
 - No chain is maintained.
- Bigger table may be needed.
 - Generally Load factor (λ) should be ≤ 0.5 .
- Collision can still occur!
 - *Alternative cells are tried until an empty cell is found.*

Open addressing

Cells $h_0(x)$, $h_1(x)$, $h_2(x)$... are tried in succession, where

$$h_i(x) = (x + f(i)) \% TableSize, \text{ with } f(0) = 0$$

- The function $f(i)$ is the collision resolution strategy.

Open addressing

Cells $h_0(x)$, $h_1(x)$, $h_2(x)$... are tried in succession, where

$$h_i(x) = (x + f(i)) \% TableSize, \text{ with } f(0) = 0$$

- The function $f(i)$ is the **collision resolution strategy**.
- Common collision resolution strategies:
 - Linear Probing
 - Quadratic Probing
 - Double Hashing

Linear Probing

- Sequentially searching the table until an empty cell is found.
 - $f(i) = i$

Linear Probing

- Sequentially searching the table until an empty cell is found.
 - $f(i) = i$
- Example:
 - Insert items with keys 89, 18, 49, 58, 9 into an empty hash table.
 - Table size is 10.

Linear Probing

- Sequentially searching the table until an empty cell is found.
 - $f(i) = i$
- Example:
 - Insert items with keys 89, 18, 49, 58, 9 into an empty hash table.
 - Table size is 10.
- Insert 89: $(89 + f(0)) \% 10 = (89 + 0) \% 10 = 9$

Linear Probing

- Sequentially searching the table until an empty cell is found.
 - $f(i) = i$
- Example:
 - Insert items with keys 89, 18, 49, 58, 9 into an empty hash table.
 - Table size is 10.
- Insert 89: $(89 + f(0)) \% 10 = (89 + 0) \% 10 = 9$
- Insert 59: $(59 + f(0)) \% 10 = (59 + 0) \% 10 = 9$
 $i = 1, (59 + f(1)) \% 10 = 59 + 1 \% 10 = 0$

Linear Probing

- Sequentially searching the table until an empty cell is found.
 - $f(i) = i$
- Example:
 - Insert items with keys 89, 18, 49, 58, 9 into an empty hash table.
 - Table size is 10.
 - Insert 89: $(89 + f(0)) \% 10 = (89 + 0) \% 10 = 9$
 - Insert 59: $(59 + f(0)) \% 10 = (59 + 0) \% 10 = 9$
 $i = 1, (59 + f(1)) \% 10 = 59 + 1 \% 10 = 0$
 - Insert 69:
 - $69 + f(0) \% 10 = 9$
 - $69 + f(1) \% 10 = 0$
 - $69 + f(2) \% 10 = 1$

Empty Table
0
1
2
3
4
5
6
7
8
9

Figure 5.11 Hash table with linear probing, after each insertion

hash (89, 10) = 9

	Empty Table	After 89
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		89

Figure 5.11 Hash table with linear probing, after each insertion

$\text{hash} (89, 10) = 9$
 $\text{hash} (18, 10) = 8$

	Empty Table	After 89	After 18
0			
1			
2			
3			
4			
5			
6			
7			
8			18
9		89	89

Figure 5.11 Hash table with linear probing, after each insertion

$\text{hash}(89, 10) = 9$
 $\text{hash}(18, 10) = 8$
 $\text{hash}(49, 10) = 9$

	Empty Table	After 89	After 18	After 49
0				49
1				
2				
3				
4				
5				
6				
7				
8			18	18
9		89	89	89

Figure 5.11 Hash table with linear probing, after each insertion

$\text{hash}(89, 10) = 9$
 $\text{hash}(18, 10) = 8$
 $\text{hash}(49, 10) = 9$
 $\text{hash}(58, 10) = 8$

	Empty Table	After 89	After 18	After 49	After 58
0				49	49
1					58
2					
3					
4					
5					
6					
7					
8			18	18	18
9		89	89	89	89

Figure 5.11 Hash table with linear probing, after each insertion

$\text{hash}(89, 10) = 9$
 $\text{hash}(18, 10) = 8$
 $\text{hash}(49, 10) = 9$
 $\text{hash}(58, 10) = 8$
 $\text{hash}(69, 10) = 9$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Figure 5.11 Hash table with linear probing, after each insertion

Linear probing (clustering problem)

- If the table is big enough, a free cell can always be found.
 - The time to find that space can get quite long!

Linear probing (clustering problem)

- If the table is big enough, a free cell can always be found.
 - The time to find that space can get quite long!
- Even if the table is relatively empty, blocks of occupied cells start forming.
 - a.k.a '*primary clustering*'.

Linear probing (clustering problem)

- If the table is big enough, a free cell can always be found.
 - The time to find that space can get quite long!
- Even if the table is relatively empty, blocks of occupied cells start forming.
 - a.k.a '*primary clustering*'.
- Any key that hashes into the cluster will require several attempts to resolve the collision.
 - Then it gets added to the cluster 😞

Analysis of insertion, search

- Avg #of cells examined in an insertion in linear probing

$$= \frac{1}{2} \left(1 + \frac{1}{(1 - \lambda)^2} \right)$$

(Proof is beyond the scope of this lecture ☹)

Analysis of insertion, search

- Avg #of cells examined in an insertion in linear probing

$$= \frac{1}{2} \left(1 + \frac{1}{(1 - \lambda)^2} \right)$$

(Proof is beyond the scope of this lecture ☹)

- If $\lambda = 0.5$, avg comparison for insertion is 2.5.
- $\lambda = 0.75$: 8.5 comparisons expected!
- $\lambda = 0.9$: the 50 !!

Quadratic probing

- Eliminates the primary clustering problem of Linear probing.
- Collision function is quadratic.
- Popular choice is $f(i)=i^2, i = 0,1,2, \dots N$
- If the hash function evaluates to 'h', and that cell is already occupied, then try cells $h + 1^2, h + 2^2, h + 3^2 \dots$

Quadratic probing

- Eliminates the primary clustering problem of Linear probing.
- Collision function is quadratic.
- Popular choice is $f(i)=i^2$, $i = 0,1,2, \dots N$
- If the hash function evaluates to 'h', and that cell is already occupied, then try cells $h + 1^2, h + 2^2, h + 3^2 \dots$
- i.e. it examines cells 1, 4, 9 and so on cells away from the original probe.

After insert 89

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	89

- Insert 89:
 - $hash_i(x) = (x + f(i)) \bmod TableSize$,
 - $f(i) = i^2, f(0) = 0, TableSize = 10$
 - $hash_0(89) = (89 + 0) \% 10 = 9$
 - No collision at index-9. So insert there.

After insert 89 After insert 18

0		
1		
2		
3		
4		
5		
6		
7		
8		18
9	89	89

- Insert 18:
 - $hash_0(18) = (18 + 0) \% 10 = 8$
 - No collision at index-8. So insert there.

	After insert 89	After insert 18	After insert 49
0			49
1			
2			
3			
4			
5			
6			
7			
8		18	18
9	89	89	89

- Insert 49:
 - $hash_0(49) = (49 + 0) \% 10 = 9$
 - **Collision** at index-9.
 - Try $i = 1, f(1) = 1^2 = 1$
 - $hash_1(49) = (49 + 1) \% 10 = 0$
 - No collision at index-0. Insert there.

	After insert 89	After insert 18	After insert 49	After insert 58
0			49	49
1				
2				58
3				
4				
5				
6				
7				
8		18	18	18
9	89	89	89	89

- Insert 58:
 - $hash_0(58) = (58 + 0) \% 10 = 8$
 - **Collision** at index-8.
 - Try $i = 1, f(1) = 1^2 = 1$
 - $hash_1(58) = (58 + 1) \% 10 = 9$
 - **Collision** at index-9.
 - Try $i = 2, f(2) = 2^2 = 4$
 - $hash_2(58) = (58 + 4) \% 10 = 2$
 - No collision at index-2. Insert there.

	After insert 89	After insert 18	After insert 49	After insert 58	After insert 9
0			49	49	49
1					
2				58	58
3					9
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

- Insert 9:
 - $hash_0(9) = (9 + 0) \% 10 = 9$
 - **Collision** at index-9.
 - Try $i = 1, f(1) = 1^2 = 1$
 - $hash_1(9) = (9 + 1) \% 10 = 0$
 - **Collision** at index-0.
 - Try $i = 2, f(2) = 2^2 = 4$
 - $hash_2(9) = (9 + 4) \% 10 = 3$
 - No collision at index-3. Insert there.

Quadratic probing

- Problem:
 - No guarantee to find an empty space if $\lambda \geq 0.5$.
 - If the hash TableSize is not prime, the problem gets much severe!

Quadratic probing

- Problem:
 - No guarantee to find an empty space if $\lambda \geq 0.5$.
 - If the hash TableSize is not prime, the problem gets much severe!
- One theorem says:
 - If the tableSize is prime and $\lambda < 0.5$, all probes will be to different locations and an item can always be inserted.

Some considerations

- What happens when λ is too high:
 - Dynamically expand the table when $\lambda > 0.5$, a.k.a 're-hashing'.
 - Double the *tableSize* to a prime number.
 - While expanding the hash table, re-insert in the new table with a new hash-function.

Some considerations

- What happens when λ is too high:
 - Dynamically expand the table when $\lambda > 0.5$, a.k.a 're-hashing'.
 - Double the *tableSize* to a prime number.
 - While expanding the hash table, re-insert in the new table with a new hash-function.
- Although Quadratic Probing solves the primary clustering, elements that hash to the same location will probe to the same alternative cells (a.k.a 'Secondary clustering').
 - Btw, this is very less likely to occur.

Double hashing

- Second hash-function is used to solve the collision.

$$f(i) = i \times hash_2(x)$$

Double hashing

- Second hash-function is used to solve the collision.

$$f(i) = i \times hash_2(x)$$

- We apply a second hash function to x and probe at a distance:
 $1 \times hash_2(x), 2 \times hash_2(x) \dots \text{and so on.}$

Double hashing

- Second hash-function is used to solve the collision.

$$f(i) = i \times hash_2(x)$$

- We apply a second hash function to x and probe at a distance:
 $1 \times hash_2(x), 2 \times hash_2(x) \dots \text{and so on.}$
- The function $hash_2(x)$ should never evaluate to Zero. Why?

Double hashing

- Second hash-function is used to solve the collision.

$$f(i) = i \times hash_2(x)$$

- We apply a second hash function to x and probe at a distance:

$$1 \times hash_2(x), 2 \times hash_2(x) \dots \text{and so on.}$$

- The function $hash_2(x)$ should never evaluate to Zero. Why?
 - E.g. let $hash_2(x) = x \bmod 9$, and try to insert 99 in the previous example.

Double hashing

- Second hash-function is used to solve the collision.

$$f(i) = i \times hash_2(x)$$

- We apply a second hash function to x and probe at a distance:

$$1 \times hash_2(x), 2 \times hash_2(x) \dots \text{and so on.}$$

- The function $hash_2(x)$ should never evaluate to Zero. Why?
 - Let $hash_2(x) = x \bmod 9$, and try to insert 99 in the previous example.
- A function such as $hash_2(x) = R - (x \bmod R)$ will work!
 - where R is a prime number $< TableSize$.
 - E.g. try $R = 7$ for the previous example. $(7 - (x \bmod 7))$.

Empty Table
0
1
2
3
4
5
6
7
8
9

- $hash_i(x) = (x + f(i)) \bmod 10$
- $f(i) = i \times hash_2(x), f(0) = 0$
- $hash_2(x) = R - (x \bmod R)$
- Let, $R = 7$
- $hash_2(x) = 7 - (x \bmod 7)$

Figure 5.18 Hash table with double hashing, after each insertion

	Empty Table	After 89
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		89

- Insert 89:
 - $hash_i(x) = (x + f(i)) \bmod 10$
 - Initially $i = 0, f(0) = 0$
 - $hash_0(89) = (89 + 0) \% 10 = 9$
 - No collision. So insert at index-9 of hash-table.

Figure 5.18 Hash table with double hashing, after each insertion

	Empty Table	After 89	After 18
0			
1			
2			
3			
4			
5			
6			
7			
8			18
9		89	89

- Insert 18:
 - $hash_0(18) = (18 + 0) \% 10 = 8$
 - No collision. So insert at index-8 of hash-table.

Figure 5.18 Hash table with double hashing, after each insertion

	Empty Table	After 89	After 18	After 49
0				
1				
2				
3				
4				
5				
6				49
7				
8			18	18
9		89	89	89

Figure 5.18 Hash table with double hashing, after each insertion

- Insert 49:
 - $hash_0(49) = (49 + 0) \% 10 = 9$
 - **Collision** at Index-9
 - Use $f(i) = i * hash_2(x)$ as collision resolution function.
- First attempt, $i = 1$.
- $hash_2(x) = 7 - (x \bmod 7)$
 $= 7 - (49 \bmod 7)$
 $= 7 - 0 = 7$
- $f(1) = 1 * hash_2(x) = 7$
- $hash(49)$
 $= (49 + f(1)) \bmod 10$
 $= (49 + 7) \% 10 = 6$
- No collision at index-6.

	Empty Table	After 89	After 18	After 49	After 58
0					
1					
2					
3					58
4					
5					
6				49	49
7					
8			18	18	18
9		89	89	89	89

Figure 5.18 Hash table with double hashing, after each insertion

- Insert 58:
 - $hash_0(58) = (58 + 0) \% 10 = 8$
 - **Collision** at index-8
 - For first attempt, $i = 1$
 - $hash_2(x) = 7 - (x \bmod 7)$
 $= 7 - (58 \bmod 7)$
 $= 7 - 2 = 5$
 - $hash(58)$
 $= (58 + f(1)) \bmod 10$
 $= (58 + 5) \% 10 = 3$
- No collision at index-3

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

Figure 5.18 Hash table with double hashing, after each insertion

- Insert 69:
 - $hash_0(69) = (69 + 0) \% 10 = 9$
 - **Collision** at index-9.
 - $hash_2(69)$
 $= 7 - (69 \bmod 7)$
 $= 7 - 6 = 1$
 - $hash(69)$
 $= (69 + f(1)) \bmod 10$
 $= (69 + 1) \% 10 = 0$
 - No collision at index-0

**** However, if collision occurred for $i = 1$, we need to recalculate $hash_2(x)$ for $i = 2$**

Summary

- Hash tables can be used to implement the insert and find operations in $O(1)$.
- It depends on the Load factor (λ), not on the number of elements (N).
- It is important to have a prime TableSize and a correct choice of Load-factor and Hash-function.
- For Separate chaining, λ should be close to 1.
- For Open addressing, λ should not exceed 0.5, unless this will slow down the entire process.
- Re-hashing can be implemented to grow (or shrink) the table.

References

- Chapter 5, *Mark Allen Weiss - Data Structures and Algorithm Analysis in C++ - 2014*
- <https://www.hackerearth.com/practice/data-structures/hash-tables/basics-of-hash-tables/tutorial/>

Thank you